

Project Overview

This project aims to create a music genre classifier from a machine-learning basis, with the goal of being able to predict one of ten genres from an audio sample. Our group approached this problem with a CNN model that classifies Mel-spectrograms of audio segments, treating them as images and thus the model as a whole as some sort of image classifier. In addition to this, we also tested and compared model ensembling with the base CNN model's performance to see if averaging three of the same model is better than relying on just one. Our results show that there is an improvement in performance when using model ensembling.

Dataset Description

Our team utilize the GTZAN Dataset, downloaded from Kaggle, specifically the .wav files within the genres_original folder. Although using the pre-made images_original folder would have been a much simpler approach for our group, generating spectrograms off of the .wav files gives our model complete flexibility with the graph's parameters, such as the number of mel bands, dB scaling, and the output resolution (in our groups case, the number of mel bands used is 128 and the output resolution is 128x256). Additionally, the pre-made images are not clean data, as they are just matplotlib screenshots. These pictures are also RGB pictures, and not actual numeric spectrogram values. Finally, as will be explained later, our largest increase in accuracy came from splitting each 30-second "song" into ten 3-second segments. Using the pre-made images are not helpful because each spectrogram image in the dataset spans the entire 30 seconds.

The dimensions of the dataset itself is 1,000 songs (10 genres x 100 tracks), each 30 seconds sampled at 22,050 Hz. Upon actually trying to use the dataset and subsequent

“research”, we discovered that the jazz.00054.wav file was corrupted, so in reality the dataset contained 999 usable tracks.

Data Preprocessing

Because our group decided to use the raw .wav files to train our model, some of the data preprocessing is just converting .wav files into a spectrogram. To start, we set up a gate to make sure that all audio files are at 22050 Hz. This was done because it was a very easy add, and it was done assuming that we were going to try and do part 9 of the project where we may have to resample the audio file (CD quality is 44,100) to fit our model.

Next, our program gets a mel spectrogram from the torchaudio library. Our window size (N_FFT) is set to 2048, which is how many audio samples go into each fast fourier transform window (upon further research/looking into documentation). A larger window analyzes a longer chunk of sound at once, giving finer frequency resolution with the trade-off of blurrier timing for the model. Our HOP_LENGTH was set to 512, which is how far the window slides. At 22,050 Hz with 512 samples, there are around 23 ms per step. Since the hop is smaller than the window itself (512 vs 2048), neighboring windows overlap each other, giving a smoother picture across time. We used 128 mel bands, meaning that the frequencies that the FFT (fast-fourier transform) gets grouped into 128 mel bands, essentially making the height of our spectrogram images. We also apply decibel scaling to compress the frequencies into something that the model can actually interpret, also mimicking human hearing, which is logarithmic.

Next, our data is segmented into 3-second units and then normalized using min-max scaling. Each 3-second segment is scaled using its own min and max. The smallest value becomes 0, and the largest becomes 1, and the values between land proportionally between the two. This is to optimize the training of the neural network, making sure that the inputs are in a

small and consistent range. A CNN needs more than 1,000 samples because the model oftentimes overfits, having a disproportionate amount of parameters compared to data points. To prevent overfitting, we segment the data. Before doing this, the model's training accuracy sat at around 90%, while the validation accuracy was at 10%, showing that the model was just random guessing, but memorizing the training dataset.

Lastly, our data's train-test split is done by track, not segment. This is done so that segments of the same song do not appear in both the training and test set, which would most certainly skew the results of the model.

Model Architecture

Initially, our input was 128x128, but was resized to 128x256 to capture more detail (it was initially much smaller because our group was using the CPU and not the T4 GPU, which made us lose out on a lot of processing power). Our CNN is built of 4 convolutional blocks, each with the following:

1. 2 Conv2D layers: each filter only covers a 3x3 patch at a time, but stacking them makes the model look at the outputs of the first, each of those having already summarized a 3x3 region. This allows each block to recognize larger structures before downsampling versus capturing only local patterns.
 - a. Having 2 Conv2Ds is better than just having one larger one because it introduces more non-linearity, as there is a ReLU activation between them.
2. 2 BatchNorms after each Conv2D: this renormalizes the outputs so the next layer receives stable and well-scaled inputs. Having two Conv2Ds requires two BatchNorms after each one so that each one can remain stable. In every batch normalization layer, the

momentum is set to 0.85. It was originally 0.99, but it was lowered to to make sure the model adapts quickly (this also helped with the validation accuracy being stuck at 0.1)

3. MaxPooling2D: this halves the resolution, shrinking the data for efficiency as well as building a hierarchy within the model, allowing the network to go from local detail to big-picture patterns. The model aims to retrieve and learn all that it can before this layer takes place.

Finally, at the very end, we use GlobalAveragePooling2D to essentially summarize each feature map into one number. Our group tried just using `.flatten()`, but this took much too long for our liking. It would not be feasible to complete the project on time using this, and GlobalAveragePooling2D significantly decreased processing time. As for the ensemble, we essentially made three of these models, each with different seeds for diversity, and averaged the results.

Evaluation and Results

The CNN's per-segment test accuracy was 75.90%, with its per-track accuracy being at 80.50%. The ensemble performed much better than the standalone CNN, with its per-segment test accuracy being at 81%, and its per-track accuracy being at 86%. The data will focus on the ensemble's data, not only because it was more accurate, but because the ensemble itself was the end goal of this project, using multiple CNNs as its "building blocks".

On a per-trag basis, classical had perfect recall, meaning the model caught all 20 classical tracks. It's precision was also perfect for metal, which points to an interesting idea that these two genres are very musically distinct as captured by the spectrogram. It is weakest when it comes to recognizing rock, and has the lowest precision score for pop. These results show that the model is using pop as its "default guess", showing that it may be broad and overlaps many styles.

Code Quality

The structure of our program is highly modular, and even though some functions are only called once, the program is broken down into blocks.

1. Load fixed waveform: loads, converts to mono, resamples
2. Spec from waveform: creates normalized spectrogram
3. Load segmented: traverse the dataset and build arrays
4. Build CNN: construct the model; made making the ensemble afterwards much easier

Some functions are helper functions for others, and said functions are labeled with a leading underscore.

Our ensemble uses controlled, repeatable randomness through the use of setting each models seed to 100 + whatever index the model is within the ensemble. Additionally, constants are set in one place so that they are easy to change and locate, even though it may seem as unnecessary variables being created. Said “global” variables are also in all caps to make sure that programmers would know that it is a global variable, and that if they want to control or change it, they can simply just look earlier in the code.

Because of the “infamous” jazz00054.wav file within the dataset, there were many try/except blocks to skip around it, also just as general use just in case a file path doesn’t exist. Additionally, within the audio normalization, there is a guard against dividing by zero. The models were also saved and could be loaded from drive.

Conclusion/Future

Creating a spectrogram CNN proved to be successful enough for this project. What made the largest difference was segmenting data and using an ensemble of three CNN models as opposed to just one. However, despite our high accuracy, larger and more complex models

proved to still struggle with the rock genre. The largest takeaway from this project into potential future projects is that data quality/quantity and normalization matters more than model size. Our team initially had a CNN that had around 1.2 million parameters, and after taking too long was reduced to 30,000, but learned at a faster rate and also may have been more accurate.

Exploring the idea of image classification and its use in music genre detection has a further downstream task/idea of an AI model being able to mimic sounds based on a certain audio sample by producing a mel-spectrogram image of the audio, and then converting that image back to a .wav file format or something of similar nature. One very interesting application of this is the use of AI and spectrogram generation/classification in the captures and NAMs and IRs for guitarists. Even just as a concept, because guitarists (and perhaps even bassists) spend a significant amount of time and money on pedals and effects to achieve a certain tone of certain artists, it would be interesting to see how much closer, if at all, an ML model can get to replicating that sound as opposed to an expansive pedal board/multifx setup.